ENSIAS

Ecole Nationale Superieure d'Informatique et d'Analyse des Systemes

DevPilot AI

Prompts complets d'implementation et de test

De la Phase 1 a la Phase 14 - Ollama-first

Realise par : Oussama Mahi
Projet : Plateforme Agentic RAG pour documents prives
Stack principal : FastAPI, PostgreSQL, Redis, Celery, Qdrant, Ollama
Objectif : Guider l'implementation phase par phase
Date : May 11, 2026

Document de travail pour implementation, tests et suivi d'avancement

Contents

Introduction **2**

1 Master Prompt **3**

2 Phase 1 : Project Structure and Docker Foundation **5**
2.1 Prompt d'implementation 5
2.2 Prompt de test manuel 7
2.3 Description du resultat attendu 7

3 Phase 2 : Backend Core Configuration **8**
3.1 Prompt d'implementation 8
3.2 Prompt de test manuel 9
3.3 Description du resultat attendu 10

4 Phase 3 : PostgreSQL, SQLAlchemy, Alembic, and Models **11**
4.1 Prompt d'implementation 11
4.2 Prompt de test manuel 13
4.3 Description du resultat attendu 14

5 Phase 4 : Authentication and JWT **15**
5.1 Prompt d'implementation 15
5.2 Prompt de test manuel 16
5.3 Description du resultat attendu 18

6 Phase 5 : Workspace Management **19**
6.1 Prompt d'implementation 19
6.2 Prompt de test manuel 20
6.3 Description du resultat attendu 21

7 Phase 6 : Document Upload **22**
7.1 Prompt d'implementation 22
7.2 Prompt de test manuel 23
7.3 Description du resultat attendu 24

8 Phase 7 : Celery and Asynchronous Document Processing **25**
8.1 Prompt d'implementation 25
8.2 Prompt de test manuel 26
8.3 Description du resultat attendu 28

9 Phase 8 : Text Cleaning and Chunking **29**
9.1 Prompt d'implementation 29
9.2 Prompt de test manuel 30
9.3 Description du resultat attendu 33

10 Phase 9 : Ollama Embeddings and Qdrant Indexing **34**
10.1 Prompt d'implementation 34
10.2 Prompt de test manuel 35
10.3 Description du resultat attendu 37

11 Phase 10 : Semantic Retrieval Endpoint **38**
11.1 Prompt d'implementation 38
11.2 Prompt de test manuel 39
11.3 Description du resultat attendu 40

12 Phase 11 : Basic RAG Chat with Ollama and Citations **41**
12.1 Prompt d'implementation 41
12.2 Prompt de test manuel 42
12.3 Description du resultat attendu 44

13 Phase 12 : Simple Evaluation of Answers **45**
13.1 Prompt d'implementation 45
13.2 Prompt de test manuel 46
13.3 Description du resultat attendu 46

14 Phase 13 : Agentic Workflow v1 **47**
14.1 Prompt d'implementation 47
14.2 Prompt de test manuel 48
14.3 Description du resultat attendu 49

15 Phase 14 : Hybrid Search **50**
15.1 Prompt d'implementation 50
15.2 Prompt de test manuel 51
15.3 Description du resultat attendu 52

16 Prompt de debogage **53**

17 Prompt de revue d'architecture **54**

Conclusion **55**

Introduction

Ce document regroupe les prompts complets a utiliser pour construire le projet **DevPilot AI** de maniere progressive. Chaque phase contient un prompt d'implementation, un prompt de test manuel, puis une description claire du resultat attendu. L'objectif est de permettre a un etudiant ou a un developpeur de reproduire le travail sans demander a l'IA de generer tout le projet d'un seul coup.

Note

La regle principale est simple : utiliser un seul prompt d'implementation, tester la phase, corriger les erreurs, puis passer a la phase suivante. Cette methode evite les architectures incoherentes et facilite la demonstration devant un encadrant.

1 Master Prompt

Objectif de la phase

Ce prompt initialise le contexte global du projet. Il doit etre donne au coding assistant avant de commencer la Phase 1.

```
You are a senior backend engineer, AI engineer, software architect, and DevOps engineer.

I want to build a final-year engineering project called:

DevPilot AI: A Secure Multi-Workspace Agentic RAG Platform for Private Document Intelligence.

The goal is to build a professional AI platform where users can:
1. Register and log in.
2. Create private workspaces.
3. Upload PDF, TXT, and Markdown documents.
4. Process documents asynchronously.
5. Extract text from documents.
6. Clean the extracted text.
7. Split text into chunks.
8. Generate embeddings locally using Ollama.
9. Store vectors in Qdrant.
10. Ask questions about uploaded documents.
11. Retrieve relevant chunks.
12. Generate answers using a local Ollama LLM.
13. Return answers with citations.
14. Store conversations.
15. Evaluate answer quality later.
16. Monitor the system later.

Important:
I do not want to use OpenAI as the default provider.
The project must be Ollama-first from the beginning.

Use this stack:
- Backend: FastAPI, Python
- Database: PostgreSQL
- Migrations: Alembic
- Auth: JWT
- Queue: Redis + Celery
- Vector database: Qdrant
- Local AI provider: Ollama
- Generation model: qwen3:8b
- Embedding model: nomic-embed-text
- Frontend later: Next.js + TypeScript
- Deployment: Docker Compose

Core architecture principles:
- Clean architecture
- Modular backend
- Environment variables
- No hardcoded secrets
- Services separated from routes
```

```
- Models separated from schemas
- Clear error handling
- Tests after every phase
- Manual verification commands after every phase

Do not build everything at once.
Work phase by phase.
After each phase, give me:
1. Files changed.
2. What was implemented.
3. Commands to run.
4. Manual tests.
5. Expected results.
6. Common errors and how to fix them.

Start by proposing the project structure and implementing Phase 1 only.
```

2 Phase 1 : Project Structure and Docker Foundation

Objectif de la phase

Creer la structure initiale du projet, les dossiers principaux, la configuration Docker Compose et un backend minimal avec FastAPI.

2.1 Prompt d'implementation

```
Implement Phase 1 of DevPilot AI: project structure and Docker foundation.

Create a clean monorepo structure:

devpilot-ai/
|-- backend/
|-- frontend/
|-- monitoring/
|-- docs/
|-- docker-compose.yml
|-- .env.example
|-- README.md
`-- Makefile

Backend structure:

backend/
|-- app/
|   |-- main.py
|   |-- core/
|   |   |-- config.py
|   |   |-- security.py
|   |   |-- logging.py
|   |   `-- exceptions.py
|   |-- db/
|   |   |-- session.py
|   |   `-- base.py
|   |-- models/
|   |-- schemas/
|   |-- services/
|   |-- api/
|   |   |-- deps.py
|   |   `-- routes/
|   |-- workers/
|   |-- utils/
|   `-- agents/
|-- tests/
|-- alembic/
|-- requirements.txt
`-- Dockerfile
```

```
Create Docker Compose services:
- postgres
- redis
- qdrant
- backend

Do not add Celery worker yet. It will come later.

Use Ollama-first configuration from the beginning.

Create .env.example with:

APP_NAME=DevPilot AI
APP_ENV=development
SECRET_KEY=change_this_secret_key

FRONTEND_URL=http://localhost:3000
BACKEND_CORS_ORIGINS=http://localhost:3000

POSTGRES_USER=devpilot
POSTGRES_PASSWORD=devpilot
POSTGRES_DB=devpilot
POSTGRES_HOST=postgres
POSTGRES_PORT=5432
DATABASE_URL=postgresql+asyncpg://devpilot:devpilot@postgres:5432/devpilot

REDIS_URL=redis://redis:6379/0
QDRANT_URL=http://qdrant:6333

LLM_PROVIDER=ollama
EMBEDDING_PROVIDER=ollama
OLLAMA_BASE_URL=http://host.docker.internal:11434
OLLAMA_GENERATION_MODEL=qwen3:8b
OLLAMA_EMBEDDING_MODEL=nomic-embed-text
GENERATION_TEMPERATURE=0.2
GENERATION_MAX_TOKENS=1000
EMBEDDING_DIMENSION=768

OPENAI_API_KEY=
OPENAI_GENERATION_MODEL=gpt-4o-mini
OPENAI_EMBEDDING_MODEL=text-embedding-3-small

Important:
- OpenAI must be optional, not required.
- Backend must start without OpenAI API key.
- Add extra_hosts to backend service:
  host.docker.internal:host-gateway
- This allows backend container to call Ollama running on host.

Create README.md explaining:
- project goal
- technologies
- how to run Docker Compose
- how to copy .env.example to .env
- how to check backend health
- how to install Ollama later

Create Makefile with:
- make up
- make down
- make logs
- make ps
- make backend-shell

For now, implement a minimal FastAPI app with /health endpoint only.
```

```
Do not implement database models yet.
Do not implement authentication yet.
Do not implement document upload yet.
```

2.2 Prompt de test manuel

```
Test Phase 1 manually.

Run:

cd ~/Documents/PFAA/devpilot-ai

cp .env.example .env

docker compose config

docker compose up -d postgres redis qdrant backend

docker compose ps

curl http://localhost:8000/health

docker compose logs backend --tail=100

Expected:
1. docker compose config works.
2. postgres, redis, qdrant, and backend are running.
3. /health returns status ok.
4. Backend starts without OpenAI API key.
5. Logs show no critical errors.

Also verify project structure:

tree -L 4

If something fails, fix:
- docker-compose.yml
- backend Dockerfile
- requirements.txt
- app/main.py
- .env.example
```

2.3 Description du resultat attendu

Description du resultat attendu

Docker Compose doit etre valide, les conteneurs principaux doivent etre en etat Up, le endpoint /health doit retourner une reponse JSON positive, et aucun message d'erreur lie a OpenAI ne doit apparaitre. La structure de dossiers doit etre claire et conforme a l'architecture prevue.

3 Phase 2 : Backend Core Configuration

Objectif de la phase

Mettre en place la configuration centralisee, les logs, CORS, les endpoints systeme et la configuration Ollama-first.

3.1 Prompt d'implementation

```
Implement Phase 2: backend core configuration.

Requirements:

1. Configuration:
Use pydantic-settings.
Create backend/app/core/config.py.
Load settings from environment variables.

Settings must include:
- APP_NAME
- APP_ENV
- SECRET_KEY
- FRONTEND_URL
- BACKEND_CORS_ORIGINS
- DATABASE_URL
- REDIS_URL
- QDRANT_URL
- LLM_PROVIDER
- EMBEDDING_PROVIDER
- OLLAMA_BASE_URL
- OLLAMA_GENERATION_MODEL
- OLLAMA_EMBEDDING_MODEL
- GENERATION_TEMPERATURE
- GENERATION_MAX_TOKENS
- EMBEDDING_DIMENSION
- OPENAI_API_KEY optional
- OPENAI_GENERATION_MODEL optional
- OPENAI_EMBEDDING_MODEL optional

Important:
- OPENAI_API_KEY must not be required.
- Ollama must be default.

2. FastAPI:
Update app/main.py.
Add:
- app title
- version
- CORS middleware
- startup log
- shutdown log
- global exception handler
```

```
- /health endpoint

3. Logging:
Create backend/app/core/logging.py.
Use structured JSON-style logs if possible.
At startup, log:
- app_name
- app_env
- postgres host or database url masked
- redis_url
- qdrant_url
- llm_provider
- embedding_provider
- ollama_base_url
- ollama_generation_model
- ollama_embedding_model
- embedding_dimension

Do not log secrets.

4. System routes:
Create:
GET /api/v1/system/ai-config

Response:
{
  "llm_provider": "ollama",
  "embedding_provider": "ollama",
  "ollama_base_url": "...",
  "generation_model": "qwen3:8b",
  "embedding_model": "nomic-embed-text",
  "embedding_dimension": 768
}

Do not expose secrets.

5. AI health:
Create:
GET /api/v1/system/ai-health

It should call:
GET {OLLAMA_BASE_URL}/api/tags

If Ollama is reachable:
return status ok.

If Ollama is not reachable:
return clean error message saying Ollama is not running or not reachable.

6. API router:
Register routes under /api/v1.

Do not implement database models yet.
Do not implement auth yet.
```

3.2 Prompt de test manuel

```
Test Phase 2 manually.

Run:

docker compose up -d postgres redis qdrant backend
```

```
Check health:

curl http://localhost:8000/health

Check AI config:

curl http://localhost:8000/api/v1/system/ai-config

Expected:
- llm_provider = ollama
- embedding_provider = ollama
- generation_model = qwen3:8b
- embedding_model = nomic-embed-text
- embedding_dimension = 768

Check logs:

docker compose logs backend --tail=100

Expected:
- logs show Ollama config
- no OpenAI API key required
- no gpt-4o-mini or text-embedding-3-small as active defaults

Test AI health:

curl http://localhost:8000/api/v1/system/ai-health

If Ollama is not running, it should return a clean error.
If Ollama is running, it should return ok.

Optional host test:

ollama serve
ollama pull qwen3:8b
ollama pull nomic-embed-text
curl http://localhost:11434/api/tags

If backend cannot reach Ollama from Docker, verify:
- OLLAMA_BASE_URL=http://host.docker.internal:11434
- backend has extra_hosts host.docker.internal:host-gateway
```

3.3 Description du resultat attendu

Description du resultat attendu

Le backend doit lire correctement les variables d'environnement, afficher la configuration active sans secrets, montrer Ollama comme fournisseur par defaut, et retourner une erreur propre si Ollama n'est pas lance. Le projet ne doit pas demander de cle OpenAI.

4 Phase 3 : PostgreSQL, SQLAlchemy, Alembic, and Models

Objectif de la phase

Construire le schema de base de donnees et le versionner avec Alembic.

4.1 Prompt d'implementation

```
Implement Phase 3: PostgreSQL, SQLAlchemy async, Alembic, and database models.

Requirements:

1. Database session:
Create backend/app/db/session.py.
Use SQLAlchemy async engine with asyncpg.
Create async session factory.
Create get_db dependency.

2. Base:
Create backend/app/db/base.py.
Create SQLAlchemy Base.

3. Alembic:
Initialize Alembic if not already initialized.
Configure Alembic to use DATABASE_URL from settings.
Make migrations compatible with async SQLAlchemy.

4. Models:
Create these SQLAlchemy models with UUID primary keys:

User:
- id
- email unique
- full_name
- password_hash
- role default "user"
- is_active default true
- created_at
- updated_at

Workspace:
- id
- name
- description nullable
- owner_id foreign key users.id
- created_at
- updated_at

WorkspaceMember:
- id
```

```
- workspace_id foreign key workspaces.id
- user_id foreign key users.id
- role
- created_at

Document:
- id
- workspace_id foreign key workspaces.id
- uploaded_by foreign key users.id
- filename
- file_type
- file_size
- status
- storage_path
- created_at
- processed_at nullable

Chunk:
- id
- document_id foreign key documents.id
- workspace_id foreign key workspaces.id
- content text
- chunk_index int
- token_count int
- metadata jsonb
- vector_id nullable unique
- created_at

Conversation:
- id
- workspace_id foreign key workspaces.id
- user_id foreign key users.id
- title nullable
- created_at
- updated_at

Message:
- id
- conversation_id foreign key conversations.id
- role
- content
- created_at

RetrievedChunk:
- id
- message_id foreign key messages.id
- chunk_id foreign key chunks.id
- score float
- rank int
- retrieval_strategy
- created_at

AgentRun:
- id
- message_id nullable foreign key messages.id
- agent_type
- status
- input jsonb nullable
- output jsonb nullable
- latency_ms nullable
- created_at

Evaluation:
- id
- message_id foreign key messages.id
```

```
- faithfulness float nullable
- relevance float nullable
- context_precision float nullable
- context_recall float nullable
- hallucination_score float nullable
- created_at

LLMUsage:
- id
- message_id foreign key messages.id
- provider
- model
- prompt_tokens
- completion_tokens
- total_tokens
- estimated_cost nullable
- latency_ms nullable
- created_at

5. Constraints and indexes:
- users.email unique
- workspace_members unique on workspace_id + user_id
- chunks unique on document_id + chunk_index
- index documents.workspace_id
- index chunks.document_id
- index chunks.workspace_id

6. Generate first migration.
7. Do not implement auth routes yet.
8. Do not implement document upload yet.
9. Do not use OpenAI anywhere.
```

4.2 Prompt de test manuel

```
Test Phase 3 manually.

Run:

docker compose up -d postgres redis qdrant backend

Run migration:

docker compose exec backend alembic upgrade head

Check tables:

docker compose exec postgres psql -U devpilot -d devpilot -c "\dt"

Expected tables:
- users
- workspaces
- workspace_members
- documents
- chunks
- conversations
- messages
- retrieved_chunks
- agent_runs
- evaluations
- llm_usage
- alembic_version

Check chunks schema:
```

```
docker compose exec postgres psql -U devpilot -d devpilot -c "\d chunks"

Expected:
- metadata jsonb
- vector_id nullable
- unique constraint on document_id, chunk_index

Check backend logs:

docker compose logs backend --tail=100

Expected:
- no database connection errors
- no OpenAI required

If migration fails:
- check Alembic env.py
- check DATABASE_URL
- check model imports in Alembic
- check SQLAlchemy JSONB import
```

4.3 Description du resultat attendu

Description du resultat attendu

La migration Alembic doit s'executer sans erreur et les tables attendues doivent apparaitre dans PostgreSQL. La table chunks doit contenir metadata en JSONB et vector_id nullable. Le backend doit rester fonctionnel apres migration.

5 Phase 4 : Authentication and JWT

Objectif de la phase

Ajouter l'inscription, la connexion, la generation de JWT et les routes protegees.

5.1 Prompt d'implementation

```
Implement Phase 4: authentication and JWT authorization.

Important:
The project is Ollama-first.
Do not require OpenAI API key.
Do not implement RAG yet.

Requirements:

1. Password security:
Use passlib bcrypt.
Create:
- hash_password(password: str) -> str
- verify_password(plain_password: str, hashed_password: str) -> bool

2. JWT:
Use python-jose.
Create:
- create_access_token(user)
- decode_access_token(token)

Token payload must include:
- sub: user id
- email
- role
- exp

3. Schemas:
Create backend/app/schemas/auth.py:
- UserRegister
- UserLogin
- TokenResponse
- UserResponse

4. Services:
Create backend/app/services/auth_service.py.
Business logic:
- register_user
- authenticate_user
- get_user_by_email

5. Dependencies:
Create backend/app/api/deps.py:
- get_current_user
```

```
- get_current_active_user
- require_admin

6. Routes:
Create backend/app/api/routes/auth.py.

Endpoints:
POST /api/v1/auth/register
POST /api/v1/auth/login
GET /api/v1/auth/me

7. Register behavior:
- validate email uniqueness
- hash password
- create user
- return user response without password_hash

8. Login behavior:
- verify email and password
- return access_token and token_type bearer

9. Security:
- never return password_hash
- wrong password returns 401
- duplicate email returns 400 or 409
- missing token returns 401

10. Tests:
Add tests if possible:
- register success
- duplicate email fails
- login success
- wrong password fails
- /me works with token
- /me fails without token

Do not implement workspaces yet.
```

5.2 Prompt de test manuel

```
Test Phase 4 manually.

Start services:

docker compose up -d postgres redis qdrant backend

Register user:

curl -X POST http://localhost:8000/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -d '{
    "email": "oussama@example.com",
    "full_name": "Oussama Mahi",
    "password": "StrongPassword123"
  }'

Expected:
- user returned
- no password_hash in response

Duplicate register:

Run the same command again.
```

```
Expected:
- 400 or 409 error

Login:

curl -X POST http://localhost:8000/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{
    "email": "oussama@example.com",
    "password": "StrongPassword123"
  }'

Expected:
- access_token returned
- token_type bearer

Save token:

TOKEN="PASTE_TOKEN_HERE"

Test /me:

curl http://localhost:8000/api/v1/auth/me \
  -H "Authorization: Bearer $TOKEN"

Expected:
- user info returned

Test /me without token:

curl http://localhost:8000/api/v1/auth/me

Expected:
- 401 Unauthorized

Test wrong password:

curl -X POST http://localhost:8000/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{
    "email": "oussama@example.com",
    "password": "WrongPassword"
  }'

Expected:
- 401 Unauthorized

Check database:

docker compose exec postgres psql -U devpilot -d devpilot -c \
"select id,email,full_name,role,is_active from users;"

Expected:
- user exists
- password_hash is not shown in API responses
```

5.3 Description du resultat attendu

Description du resultat attendu

Un utilisateur doit pouvoir s'inscrire et se connecter. L'API doit retourner un access_token utilisable sur /auth/me. Les erreurs de mot de passe, doublon email et token manquant doivent etre gerees proprement. Le hash du mot de passe ne doit jamais apparaitre dans les reponses API.

6 Phase 5 : Workspace Management

Objectif de la phase

Permettre aux utilisateurs authentifies de creer, lister et securiser des workspaces.

6.1 Prompt d'implementation

```
Implement Phase 5: workspace management.

Important:
- Use existing JWT authentication.
- User must be authenticated to access workspaces.
- Do not implement document upload yet.

Requirements:

1. Schemas:
Create backend/app/schemas/workspace.py:
- WorkspaceCreate
- WorkspaceUpdate
- WorkspaceResponse
- WorkspaceMemberResponse

2. Services:
Create backend/app/services/workspace_service.py.

Implement:
- create_workspace
- list_user_workspaces
- get_workspace_for_user
- update_workspace
- delete_workspace
- is_workspace_member
- require_workspace_access
- require_workspace_owner_or_admin

3. Routes:
Create backend/app/api/routes/workspaces.py.

Endpoints:
POST /api/v1/workspaces
GET /api/v1/workspaces
GET /api/v1/workspaces/{workspace_id}
PATCH /api/v1/workspaces/{workspace_id}
DELETE /api/v1/workspaces/{workspace_id}

4. Behavior:
- Authenticated user can create workspace.
- Creator becomes owner.
- Create row in workspace_members with role owner.
- User can list only workspaces where they are a member.
```

```
- User can access only their workspaces.
- Only owner or admin can update/delete workspace.
- Another user must receive 403 or 404.

5. Tests:
Add tests if possible:
- create workspace
- list workspaces
- get workspace
- update as owner
- reject access from another user
- reject update from another user

Do not implement document upload yet.
```

6.2 Prompt de test manuel

```
Test Phase 5 manually.

Start services:

docker compose up -d postgres redis qdrant backend

Login user A:

curl -X POST http://localhost:8000/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{
    "email": "oussama@example.com",
    "password": "StrongPassword123"
  }'

TOKEN_A="PASTE_TOKEN_A_HERE"

Create workspace as user A:

curl -X POST http://localhost:8000/api/v1/workspaces \
  -H "Authorization: Bearer $TOKEN_A" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "DevPilot Workspace",
    "description": "Workspace for manual testing"
  }'

WORKSPACE_ID="PASTE_WORKSPACE_ID_HERE"

List workspaces:

curl http://localhost:8000/api/v1/workspaces \
  -H "Authorization: Bearer $TOKEN_A"

Expected:
- workspace appears

Get workspace:

curl http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID \
  -H "Authorization: Bearer $TOKEN_A"

Expected:
- workspace details

Update workspace:
```

```
curl -X PATCH http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID \
  -H "Authorization: Bearer $TOKEN_A" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Updated DevPilot Workspace"
  }'

Expected:
- updated name

Create user B and login as user B.

User B tries to access user A workspace:

curl http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID \
  -H "Authorization: Bearer $TOKEN_B"

Expected:
- 403 or 404

Check database:

docker compose exec postgres psql -U devpilot -d devpilot -c \
"select workspace_id,user_id,role from workspace_members;"

Expected:
- user A is owner
- user B is not member
```

6.3 Description du resultat attendu

Description du resultat attendu

L'utilisateur A doit creer et modifier son workspace. L'utilisateur B ne doit pas pouvoir lire ou modifier ce workspace. La table workspace_members doit montrer que le createur est owner.

7 Phase 6 : Document Upload

Objectif de la phase

Ajouter l'upload securise de fichiers PDF, TXT et Markdown dans un workspace.

7.1 Prompt d'implementation

```
Implement Phase 6: document upload and document management.

Important:
- Use existing authentication and workspace access control.
- Do not implement Celery processing yet.
- Do not extract text yet.
- Do not create chunks yet.

Requirements:

1. Supported file types:
- .txt
- .md
- .pdf

2. Settings:
Add if not already present:
- MAX_UPLOAD_SIZE_MB
- UPLOAD_DIR
- FILE_ALLOWED_TYPES

Default:
MAX_UPLOAD_SIZE_MB=20
UPLOAD_DIR=/app/storage
FILE_ALLOWED_TYPES=.pdf,.txt,.md

3. Schemas:
Create backend/app/schemas/document.py:
- DocumentResponse
- DocumentStatusResponse

4. Services:
Create backend/app/services/document_service.py.

Implement:
- validate_file_type
- validate_file_size
- sanitize_filename
- save_uploaded_file
- create_document_record
- list_workspace_documents
- get_document_for_user
- get_document_status
- delete_document
```

```
5. Storage:
Save uploaded files under:

/app/storage/{workspace_id}/{document_id}/{safe_filename}

6. Routes:
Create backend/app/api/routes/documents.py.

Endpoints:
POST /api/v1/workspaces/{workspace_id}/documents
GET /api/v1/workspaces/{workspace_id}/documents
GET /api/v1/documents/{document_id}
GET /api/v1/documents/{document_id}/status
DELETE /api/v1/documents/{document_id}

7. Behavior:
- user must belong to workspace
- upload valid PDF/TXT/MD only
- invalid extension returns 400
- oversized file returns 413 or 400
- create document with status "queued"
- return document metadata
- user cannot access another user's document

8. Security:
- prevent path traversal
- never trust original filename
- sanitize filename
- do not allow .exe or unsupported types

9. Docker:
Mount storage volume for backend:
uploaded_files:/app/storage

Do not add Celery worker yet.
```

7.2 Prompt de test manuel

```
Test Phase 6 manually.

Start services:

docker compose up -d postgres redis qdrant backend

Login and save TOKEN.

Create workspace and save WORKSPACE_ID.

Create sample file:

echo "DevPilot AI document upload test." > sample.txt

Upload file:

curl -X POST http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/documents \
  -H "Authorization: Bearer $TOKEN" \
  -F "file=@sample.txt"

Expected:
- document created
- status queued
- filename sample.txt
```

```
DOCUMENT_ID="PASTE_DOCUMENT_ID_HERE"

Check status:

curl http://localhost:8000/api/v1/documents/$DOCUMENT_ID/status \
  -H "Authorization: Bearer $TOKEN"

Expected:
- status queued

List documents:

curl http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/documents \
  -H "Authorization: Bearer $TOKEN"

Expected:
- sample.txt appears

Check database:

docker compose exec postgres psql -U devpilot -d devpilot -c \
"select id,filename,file_type,status,storage_path from documents order by created_at desc limit 5;"

Check file storage:

docker compose exec backend find /app/storage -type f

Test invalid file:

echo "bad" > virus.exe

curl -X POST http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/documents \
  -H "Authorization: Bearer $TOKEN" \
  -F "file=@virus.exe"

Expected:
- 400 Bad Request

Test access isolation:
Login as user B and try to access user A document.

Expected:
- 403 or 404
```

7.3 Description du resultat attendu

Description du resultat attendu

Un fichier valide doit etre stocke physiquement et enregistre en base avec status queued. Un fichier .exe doit etre refuse. Un autre utilisateur ne doit pas pouvoir acceder au document.

8 Phase 7 : Celery and Asynchronous Document Processing

Objectif de la phase

Ajouter le worker Celery pour traiter les documents en arriere-plan.

8.1 Prompt d'implementation

```
Implement Phase 7: asynchronous document processing with Celery and Redis.

Important:
- Do not implement chunking yet.
- Do not implement embeddings yet.
- Do not call Ollama yet.
- Do not use Qdrant yet.
- Only extract text and update document status.

Requirements:

1. Celery setup:
Create backend/app/workers/celery_app.py.
Use REDIS_URL as broker and backend.

2. Celery task:
Create process_document_task(document_id: str).

3. Docker Compose:
Add service:
celery_worker

It must:
- use the same backend image
- run celery worker command
- depend on redis, postgres
- mount the same uploaded_files volume as backend
- have same environment variables as backend

4. Upload integration:
When a document is uploaded:
- create document with status queued
- send Celery task process_document_task(document_id)
- return response immediately

5. Task behavior:
- load document from PostgreSQL
- set status = processing
- read file from storage_path
- extract text depending on file type:
  .txt = read text
  .md = read text
```

```
  .pdf = use pypdf
- log extracted text length
- if extraction succeeds and text is not empty:
  status = indexed
  processed_at = current UTC datetime
- if extraction fails:
  status = failed
  processed_at = current UTC datetime
  log error clearly

6. Retry:
Add retry up to 3 times for transient errors.

7. Logs:
Celery logs should show:
- task received
- document id
- filename
- status changes
- extracted text length
- success or failure

8. Important:
If text extraction returns empty text, mark document as failed.

Do not create chunks in this phase.
```

8.2 Prompt de test manuel

```
Test Phase 7 manually.

Start services:

docker compose up -d postgres redis qdrant backend celery_worker

Check containers:

docker compose ps

Check Redis:

docker compose exec redis redis-cli ping

Expected:
PONG

Watch Celery logs in another terminal:

docker compose logs -f celery_worker

Login and create workspace.

Create TXT file:

cat > phase7_test.txt << 'EOF'
DevPilot AI is a private document intelligence platform.
This file is used to test Celery asynchronous document processing.
EOF

Upload:

curl -X POST http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/documents \
  -H "Authorization: Bearer $TOKEN" \
```

```
  -F "file=@phase7_test.txt"

DOCUMENT_ID="PASTE_DOCUMENT_ID_HERE"

Check status immediately:

curl http://localhost:8000/api/v1/documents/$DOCUMENT_ID/status \
  -H "Authorization: Bearer $TOKEN"

Expected:
- queued, processing, or indexed

Wait:

sleep 3

Check status again:

curl http://localhost:8000/api/v1/documents/$DOCUMENT_ID/status \
  -H "Authorization: Bearer $TOKEN"

Expected:
- indexed

Check database:

docker compose exec postgres psql -U devpilot -d devpilot -c \
"select id,filename,status,processed_at from documents where id = '$DOCUMENT_ID';"

Expected:
- status indexed
- processed_at not null

Test corrupted PDF:

echo "this is not a real pdf" > broken.pdf

curl -X POST http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/documents \
  -H "Authorization: Bearer $TOKEN" \
  -F "file=@broken.pdf"

BROKEN_DOCUMENT_ID="PASTE_BROKEN_DOCUMENT_ID_HERE"

sleep 3

curl http://localhost:8000/api/v1/documents/$BROKEN_DOCUMENT_ID/status \
  -H "Authorization: Bearer $TOKEN"

Expected:
- failed

Check Celery logs:
- task received
- extraction logs
- success or failure logs
```

8.3 Description du resultat attendu

Description du resultat attendu

L'upload doit retourner rapidement, puis le worker doit traiter le document en arriere-plan. Le statut doit passer a indexed pour un fichier valide et failed pour un PDF corrompu. processed_at doit etre rempli.

9 Phase 8 : Text Cleaning and Chunking

Objectif de la phase

Nettoyer le texte extrait, le decouper en chunks et sauvegarder ces chunks dans PostgreSQL.

9.1 Prompt d'implementation

```
Implement Phase 8: text cleaning and chunking.

Important:
- Use existing Celery task.
- Do not implement embeddings yet.
- Do not call Ollama yet.
- Do not store vectors in Qdrant yet.
- Only clean text, split into chunks, and save chunks in PostgreSQL.

Requirements:

1. Text cleaner:
Create backend/app/utils/text_cleaner.py.

Function:
clean_text(text: str) -> str

Cleaning rules:
- normalize repeated spaces
- normalize repeated empty lines
- remove leading/trailing whitespace
- preserve headings
- preserve punctuation
- do not destroy Markdown structure
- do not destroy code blocks too aggressively

2. Chunker:
Create backend/app/utils/chunker.py.

Function:
chunk_text(text: str, chunk_size: int = 800, overlap: int = 150) -> list[dict]

Each chunk dict:
{
  "content": "...",
  "chunk_index": 0,
  "token_count": 123,
  "metadata": {
    "start_char": 0,
    "end_char": 800
  }
}

3. Token count:
```

```
Use tiktoken if available.
If not available, use word count fallback.

4. Update Celery task:
After extracting text:
- clean text
- if cleaned text is empty, mark document failed
- delete old chunks for document if they exist
- split cleaned text into chunks
- save chunks in PostgreSQL
- set document status indexed
- set processed_at current UTC datetime

5. Database:
Use existing chunks table:
- document_id
- workspace_id
- content
- chunk_index
- token_count
- metadata
- vector_id null
- created_at

Important:
- vector_id remains null in Phase 8.
- vector_id will be filled in Phase 9.

6. Logs:
Celery logs should show:
- extracted text length
- cleaned text length
- number of chunks created
- document indexed

7. Error handling:
If chunking fails:
- status failed
- processed_at set
- no partial duplicate chunks if possible

8. Duplicate prevention:
If the document is reprocessed, delete old chunks first.
Do not create duplicate chunks.

9. Tests:
Add tests if possible:
- clean_text normalizes whitespace
- chunk_text creates chunks
- chunk_index starts at 0
- token_count exists
- uploaded document creates chunks
- broken PDF creates zero chunks
```

9.2 Prompt de test manuel

```
Test Phase 8 manually.

Start services:

docker compose up -d postgres redis qdrant backend celery_worker

Check Redis:
```

```
docker compose exec redis redis-cli ping

Expected:
PONG

Watch Celery logs:

docker compose logs -f celery_worker

Login and create workspace.

Create long test file:

cat > phase8_long_test.txt << 'EOF'
DevPilot AI is a private document intelligence platform.

It allows users to upload private documents, process them asynchronously, and prepare them for
    intelligent retrieval.

The platform uses FastAPI for the backend API, PostgreSQL for metadata, Redis and Celery for
    asynchronous processing, and Qdrant for vector search in future phases.

The goal of Phase 8 is to clean extracted text and split it into chunks.

Chunking is important because large documents cannot be sent entirely to a language model. Instead,
     the document is divided into smaller pieces.

Each chunk should have content, chunk_index, token_count, and metadata.

The metadata should include start_char and end_char if implemented.

The first chunk should start with chunk_index zero.

The next chunks should continue with increasing indexes.

Overlap is useful because it preserves context between neighboring chunks.

DevPilot AI should save chunks in the PostgreSQL chunks table.

Later, in Phase 9, these chunks will be converted into embeddings and stored in Qdrant.
EOF

Upload:

curl -X POST http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/documents \
  -H "Authorization: Bearer $TOKEN" \
  -F "file=@phase8_long_test.txt"

DOCUMENT_ID="PASTE_DOCUMENT_ID_HERE"

Wait:

sleep 3

Check status:

curl http://localhost:8000/api/v1/documents/$DOCUMENT_ID/status \
  -H "Authorization: Bearer $TOKEN"

Expected:
- indexed

Check document in database:
```

```
docker compose exec postgres psql -U devpilot -d devpilot -c \
"select id,filename,status,processed_at from documents where id = '$DOCUMENT_ID';"

Expected:
- status indexed
- processed_at not null

Check chunks count:

docker compose exec postgres psql -U devpilot -d devpilot -c \
"select count(*) from chunks where document_id = '$DOCUMENT_ID';"

Expected:
- count > 0

Inspect chunks:

docker compose exec postgres psql -U devpilot -d devpilot -c \
"select chunk_index, token_count, left(content, 120) as preview from chunks where document_id = '
    $DOCUMENT_ID' order by chunk_index;"

Expected:
- chunk_index starts at 0
- token_count not null
- content preview not empty

Check metadata:

docker compose exec postgres psql -U devpilot -d devpilot -c \
"select chunk_index, metadata from chunks where document_id = '$DOCUMENT_ID' order by chunk_index
    limit 5;"

Expected:
- metadata exists
- contains start_char/end_char or useful chunk information

Test Markdown and dirty text similarly.

Test broken PDF:

echo "this is not a valid pdf" > phase8_broken.pdf

Upload it, wait, and verify:
- status failed
- zero chunks for broken document

Phase 8 passes if:
- valid TXT becomes indexed
- valid MD becomes indexed
- broken PDF becomes failed
- valid docs create chunks
- chunks have content
- chunks have token_count
- chunks have metadata
- vector_id is still null
```

9.3 Description du resultat attendu

Description du resultat attendu

Les documents valides doivent creer des chunks en base. Chaque chunk doit avoir un index, un contenu, un token_count et des metadata. Les documents invalides doivent passer a failed et ne pas creer de chunks. vector_id doit rester null car les embeddings ne sont pas encore generes.

10 Phase 9 : Ollama Embeddings and Qdrant Indexing

Objectif de la phase

Generer des embeddings locaux avec Ollama et stocker les vecteurs dans Qdrant.

10.1 Prompt d'implementation

```
Implement Phase 9: Ollama embeddings and Qdrant vector indexing.

Important:
Now we start using Ollama and Qdrant.
Do not implement chat yet.
Do not implement answer generation yet.
Only generate embeddings for chunks and store them in Qdrant.

Requirements:

1. Ollama embedding provider:
Create backend/app/providers/base.py.
Create:
- BaseEmbeddingProvider
- EmbeddingResponse if useful

Create backend/app/providers/ollama_provider.py.
Implement:
- OllamaEmbeddingProvider

Use:
POST {OLLAMA_BASE_URL}/api/embed

Request:
{
  "model": OLLAMA_EMBEDDING_MODEL,
  "input": text
}

Support both single input and batch if possible.

2. Settings:
Use:
- EMBEDDING_PROVIDER=ollama
- OLLAMA_BASE_URL
- OLLAMA_EMBEDDING_MODEL
- EMBEDDING_DIMENSION

3. Qdrant service:
Create backend/app/services/vector_store_service.py.

Functions:
```

```
- ensure_collection()
- upsert_chunk_vector(chunk, embedding)
- upsert_chunks(document_id)
- search(workspace_id, query_vector, top_k)

Collection name:
devpilot_chunks

Vector size:
EMBEDDING_DIMENSION

Distance:
Cosine

Payload:
- workspace_id
- document_id
- chunk_id
- chunk_index
- filename

4. Update Celery task:
After chunks are saved:
- generate embeddings for each chunk using Ollama
- ensure Qdrant collection exists
- store each vector in Qdrant
- save vector_id in chunks table
- set document status indexed only after vectors are stored successfully

5. Error handling:
If Ollama is not running:
- document status failed
- log clear message
- tell user to run ollama serve and pull model

If Qdrant is not reachable:
- document status failed
- log clear message

6. System endpoints:
Add:
GET /api/v1/system/vector-health

It should check Qdrant collection status.

7. Tests:
Add tests with fake embedding provider if possible.

Do not implement retrieval endpoint yet.
```

10.2 Prompt de test manuel

```
Test Phase 9 manually.

First check Ollama on host:

curl http://localhost:11434/api/tags

If not running:

ollama serve

Pull embedding model:
```

```
ollama pull nomic-embed-text

Test Ollama embedding:

curl http://localhost:11434/api/embed \
  -H "Content-Type: application/json" \
  -d '{
    "model": "nomic-embed-text",
    "input": "DevPilot AI is a private document intelligence platform."
  }'

Expected:
- embeddings returned

Start services:

docker compose up -d postgres redis qdrant backend celery_worker

Check Qdrant:

curl http://localhost:6333/collections

Login and create workspace as before.

Upload TXT file:

echo "DevPilot AI uses FastAPI, PostgreSQL, Redis, Celery, Ollama and Qdrant." > phase9_test.txt

curl -X POST http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/documents \
  -H "Authorization: Bearer $TOKEN" \
  -F "file=@phase9_test.txt"

DOCUMENT_ID="PASTE_DOCUMENT_ID_HERE"

Wait:

sleep 5

Check status:

curl http://localhost:8000/api/v1/documents/$DOCUMENT_ID/status \
  -H "Authorization: Bearer $TOKEN"

Expected:
- indexed

Check chunks vector_id:

docker compose exec postgres psql -U devpilot -d devpilot -c \
"select chunk_index, vector_id from chunks where document_id = '$DOCUMENT_ID' order by chunk_index
    ;"

Expected:
- vector_id not null

Check Qdrant collection:

curl http://localhost:6333/collections/devpilot_chunks

Expected:
- collection exists
- vector size = 768

Check Celery logs:
```

```
docker compose logs celery_worker --tail=200

Expected:
- chunks created
- embeddings generated
- vectors stored in Qdrant

If failure:
- verify Ollama is running
- verify OLLAMA_BASE_URL=http://host.docker.internal:11434
- verify backend and celery_worker have extra_hosts
- verify Qdrant URL is http://qdrant:6333
```

10.3 Description du resultat attendu

Description du resultat attendu

Ollama doit retourner des embeddings, Qdrant doit contenir la collection devpilot_chunks, et chaque chunk doit avoir un vector_id non nul. L'indexation complete signifie maintenant : extraction, cleaning, chunking, embedding, stockage Qdrant.

11 Phase 10 : Semantic Retrieval Endpoint

Objectif de la phase

Ajouter la recherche semantique sur les vecteurs stockes dans Qdrant.

11.1 Prompt d'implementation

```
Implement Phase 10: semantic retrieval endpoint.

Important:
- Use existing Ollama embeddings and Qdrant vectors.
- Do not implement chat generation yet.
- Only retrieve relevant chunks.

Requirements:

1. Retrieval service:
Create backend/app/services/retrieval_service.py.

Function:
retrieve_semantic(workspace_id, query, top_k=5)

Flow:
- generate query embedding using Ollama
- search Qdrant collection devpilot_chunks
- filter by workspace_id
- get matching chunk IDs from payload
- fetch chunk content from PostgreSQL
- return ranked results

2. Schema:
Create retrieval schemas:
- RetrievalRequest
- RetrievedChunkResponse

Request:
{
  "query": "What is DevPilot AI?",
  "top_k": 5
}

Response result:
{
  "chunk_id": "...",
  "document_id": "...",
  "filename": "...",
  "content": "...",
  "chunk_index": 0,
  "score": 0.82,
  "metadata": {}
}
```

```
3. Route:
POST /api/v1/workspaces/{workspace_id}/retrieval/search

4. Security:
- user must belong to workspace
- Qdrant filter must include workspace_id
- user cannot retrieve chunks from another workspace

5. Tests:
- retrieval returns chunks
- top_k works
- workspace isolation works

Do not implement chat yet.
```

11.2 Prompt de test manuel

```
Test Phase 10 manually.

Make sure Phase 9 document is indexed and chunks have vector_id.

Call retrieval:

curl -X POST http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/retrieval/search \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "query": "What technologies does DevPilot AI use?",
    "top_k": 5
  }'

Expected:
- results array not empty
- each result has content
- each result has score
- each result has filename
- each result belongs to same workspace

Test top_k:

curl -X POST http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/retrieval/search \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "query": "FastAPI PostgreSQL Redis Celery",
    "top_k": 2
  }'

Expected:
- maximum 2 results

Test isolation:
Login as another user and call same endpoint.

Expected:
- 403 or 404

If no results:
- check chunks have vector_id
- check Qdrant collection has points
- check workspace_id payload in Qdrant
- check Ollama embedding model dimension
```

11.3 Description du resultat attendu

Description du resultat attendu

La recherche doit retourner les chunks les plus pertinents du workspace courant uniquement. Le nombre de resultats doit respecter top_k et l'isolation entre utilisateurs doit rester active.

12 Phase 11 : Basic RAG Chat with Ollama and Citations

Objectif de la phase

Generer une reponse a partir des chunks retrouves et retourner des citations.

12.1 Prompt d'implementation

```
Implement Phase 11: basic RAG chat with Ollama generation and citations.

Important:
- Use local Ollama for generation.
- Do not use OpenAI.
- Use semantic retrieval from Phase 10.
- Return citations.

Requirements:

1. Ollama LLM provider:
Extend backend/app/providers/ollama_provider.py.
Implement:
OllamaLLMProvider.generate(prompt)

Use:
POST {OLLAMA_BASE_URL}/api/chat

Request:
{
  "model": OLLAMA_GENERATION_MODEL,
  "messages": [
    {"role": "system", "content": "..."},
    {"role": "user", "content": "..."}
  ],
  "stream": false,
  "options": {
    "temperature": GENERATION_TEMPERATURE,
    "num_predict": GENERATION_MAX_TOKENS
  }
}

2. Prompt:
System prompt:

You are DevPilot AI, a private-document assistant.
Answer the user's question using ONLY the provided context.
If the answer is not in the context, say:
"I could not find this information in the uploaded documents."
Always cite sources using [1], [2], [3].
Do not invent facts.
Answer in the same language as the user.
```

```
3. Context format:
For each retrieved chunk:

[1] Source: filename | Chunk: index
Content:
...

4. Chat route:
POST /api/v1/workspaces/{workspace_id}/chat/query

Request:
{
  "question": "What is DevPilot AI?",
  "conversation_id": null
}

Flow:
- verify workspace access
- create conversation if conversation_id is null
- save user message
- retrieve top_k chunks
- build context with citation numbers
- call Ollama generation model
- save assistant message
- save retrieved_chunks
- save llm_usage if possible
- return answer, citations, conversation_id, message_id

5. Citations:
Each citation:
{
  "id": 1,
  "document_id": "...",
  "filename": "...",
  "chunk_id": "...",
  "chunk_index": 0,
  "score": 0.83
}

6. Routes:
GET /api/v1/workspaces/{workspace_id}/conversations
GET /api/v1/conversations/{conversation_id}

7. Security:
- user can access only own workspace conversations

8. Tests:
- ask question returns answer
- citations returned
- conversation saved
- unrelated question refuses if context insufficient

Do not implement evaluation yet.
```

12.2 Prompt de test manuel

```
Test Phase 11 manually.

Make sure Ollama is running:

curl http://localhost:11434/api/tags
```

```
Pull generation model if needed:

ollama pull qwen3:8b

Test direct generation:

curl http://localhost:11434/api/chat \
  -H "Content-Type: application/json" \
  -d '{
    "model": "qwen3:8b",
    "messages": [
      {"role": "user", "content": "Explain RAG in one short paragraph."}
    ],
    "stream": false
  }'

Start services:

docker compose up -d postgres redis qdrant backend celery_worker

Upload and index a document if needed.

Ask RAG question:

curl -X POST http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/chat/query \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "question": "What is DevPilot AI?"
  }'

Expected:
- answer returned
- citations returned
- conversation_id returned
- message_id returned
- answer uses [1] or citations

Ask unrelated question:

curl -X POST http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/chat/query \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "question": "Who won the World Cup in 1998?"
  }'

Expected:
- answer should say information not found in uploaded documents

Check database:

docker compose exec postgres psql -U devpilot -d devpilot -c \
"select role, left(content,100) from messages order by created_at desc limit 10;"

docker compose exec postgres psql -U devpilot -d devpilot -c \
"select message_id, chunk_id, score, rank from retrieved_chunks order by created_at desc limit 10;"

If generation fails:
- verify Ollama running
- verify qwen3:8b pulled
- verify OLLAMA_BASE_URL from container
```

12.3 Description du resultat attendu

Description du resultat attendu

Le chat doit produire une reponse basee uniquement sur les documents indexes. La reponse doit inclure des citations et sauvegarder les messages en base. Une question hors contexte doit etre refusee au lieu d'etre hallucinee.

13 Phase 12 : Simple Evaluation of Answers

Objectif de la phase

Evaluer la qualite des reponses generees et sauvegarder les scores.

13.1 Prompt d'implementation

```
Implement Phase 12: answer evaluation.

Important:
- Keep it simple.
- Use Ollama for LLM-based evaluation if available.
- Provide heuristic fallback.

Requirements:

1. Evaluation service:
Create backend/app/services/evaluation_service.py.

Function:
evaluate_answer(question, answer, contexts)

Return:
- faithfulness
- relevance
- context_precision
- hallucination_score
- explanation

2. LLM evaluator:
Use Ollama to judge answer against context.
Return strict JSON.

3. Fallback heuristic:
If Ollama evaluator fails:
- estimate relevance by word overlap
- hallucination_score low if answer says information not found
- faithfulness moderate if answer contains citation markers

4. Update chat flow:
After generating answer:
- evaluate answer
- save evaluation row in PostgreSQL
- return evaluation in chat response

5. Route:
GET /api/v1/messages/{message_id}/evaluation

6. Security:
- user can access evaluation only if message belongs to their conversation/workspace
```

```
7. Tests:
- evaluation saved
- evaluation returned
- unauthorized access rejected
```

13.2 Prompt de test manuel

```
Test Phase 12 manually.

Ask a document-related question:

curl -X POST http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/chat/query \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "question": "What technologies does DevPilot AI use?"
  }'

Expected response includes:
- answer
- citations
- evaluation

Save message_id:

MESSAGE_ID="PASTE_MESSAGE_ID_HERE"

Get evaluation:

curl http://localhost:8000/api/v1/messages/$MESSAGE_ID/evaluation \
  -H "Authorization: Bearer $TOKEN"

Expected:
- faithfulness
- relevance
- hallucination_score

Check database:

docker compose exec postgres psql -U devpilot -d devpilot -c \
"select faithfulness,relevance,hallucination_score from evaluations order by created_at desc limit
    5;"

Ask unrelated question and check hallucination behavior.

Expected:
- if answer refuses, hallucination_score should be low
```

13.3 Description du resultat attendu

Description du resultat attendu

Chaque reponse doit produire une evaluation sauvegardee en base. Les scores doivent aider a verifier la fidelite, la pertinence et le risque d'hallucination. L'acces aux evaluations doit respecter la securite du workspace.

14 Phase 13 : Agentic Workflow v1

Objectif de la phase

Remplacer le flux RAG lineaire par un workflow compose d'agents simples.

14.1 Prompt d'implementation

```
Implement Phase 13: simple Agentic RAG workflow v1.

Important:
Do not use LangGraph yet.
Build simple Python classes first.

Agents:
1. RouterAgent
2. QueryRewriterAgent
3. RetrievalAgent
4. GeneratorAgent
5. EvaluatorAgent
6. CorrectorAgent

Workflow:
User question
-> RouterAgent
-> QueryRewriterAgent
-> RetrievalAgent
-> GeneratorAgent
-> EvaluatorAgent
-> CorrectorAgent if needed
-> Final answer

RouterAgent:
- classify query_type:
  factual_question
  summary_question
  comparison_question
  technical_question
  unknown
- choose retrieval_strategy:
  semantic for now

QueryRewriterAgent:
- improve unclear queries
- can be simple rule-based first

RetrievalAgent:
- call retrieval_service

GeneratorAgent:
- call Ollama LLM provider
```

```
EvaluatorAgent:
- call evaluation_service

CorrectorAgent:
- if faithfulness < 0.7 or relevance < 0.7:
  return stricter answer or insufficient context message

AgentRun logging:
For each agent, save:
- message_id
- agent_type
- status
- input JSON
- output JSON
- latency_ms

Update chat endpoint to use the agent workflow.

Tests:
- agent runs saved
- final answer returned
- weak answer triggers corrector
- citations preserved
```

14.2 Prompt de test manuel

```
Test Phase 13 manually.

Ask question:

curl -X POST http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/chat/query \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "question": "Explain the architecture of DevPilot AI."
  }'

Expected:
- answer returned
- citations returned
- evaluation returned

Check agent runs:

docker compose exec postgres psql -U devpilot -d devpilot -c \
"select agent_type,status,latency_ms from agent_runs order by created_at desc limit 20;"

Expected agent types:
- router
- query_rewriter
- retrieval
- generator
- evaluator
- maybe corrector

Ask vague question:

curl -X POST http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/chat/query \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "question": "Explain it"
  }'
```

```
Expected:
- query rewriter handles or asks for clarification
- no crash

Ask unrelated question:
Expected:
- answer refuses if context insufficient
- corrector may be triggered
```

14.3 Description du resultat attendu

Description du resultat attendu

Le systeme doit enregistrer une trace pour chaque agent. La reponse finale doit rester avec citations, et les questions faibles ou hors contexte doivent activer la logique de correction ou de refus.

15 Phase 14 : Hybrid Search

Objectif de la phase

Combiner la recherche semantique Qdrant avec la recherche textuelle PostgreSQL.

15.1 Prompt d'implementation

```
Implement Phase 14: hybrid search.

Current retrieval is semantic using Qdrant.
Add keyword search using PostgreSQL full-text search.

Requirements:

1. Keyword search:
Search chunks.content using PostgreSQL.
Filter by workspace_id.
Return ranked chunks.

2. Semantic search:
Use existing Qdrant search.

3. Hybrid search:
Run semantic and keyword search.
Merge results using Reciprocal Rank Fusion.

RRF:
score = sum(1 / (k + rank))
Use k = 60.

4. Retrieval service:
Support:
strategy = semantic | keyword | hybrid

5. Route:
Update retrieval endpoint to accept strategy:

{
  "query": "...",
  "top_k": 5,
  "strategy": "hybrid"
}

6. RouterAgent:
- exact terms -> keyword or hybrid
- normal question -> semantic or hybrid
- technical question -> hybrid

7. Store retrieval_strategy in retrieved_chunks.

Tests:
```

```
- keyword finds exact terms
- hybrid merges results
- workspace isolation still works
```

15.2 Prompt de test manuel

```
Test Phase 14 manually.

Upload document with specific terms:

echo "DevPilot AI uses Qdrant for vector search and Celery for asynchronous document processing." >
     hybrid_test.txt

Upload and wait until indexed with vectors.

Test semantic:

curl -X POST http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/retrieval/search \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "query": "document processing system",
    "top_k": 5,
    "strategy": "semantic"
  }'

Test keyword:

curl -X POST http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/retrieval/search \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "query": "Qdrant Celery",
    "top_k": 5,
    "strategy": "keyword"
  }'

Test hybrid:

curl -X POST http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/retrieval/search \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "query": "How does DevPilot use Qdrant and Celery?",
    "top_k": 5,
    "strategy": "hybrid"
  }'

Expected:
- keyword finds exact terms
- semantic finds related meaning
- hybrid gives best combined result

Ask chat question and verify retrieved_chunks.retrieval_strategy is hybrid.

docker compose exec postgres psql -U devpilot -d devpilot -c \
"select retrieval_strategy, score, rank from retrieved_chunks order by created_at desc limit 10;"
```

15.3 Description du resultat attendu

Description du resultat attendu

La recherche keyword doit trouver les mots exacts, la recherche semantic doit trouver le sens, et hybrid doit combiner les deux. La strategie utilisee doit etre sauvegardee dans retrieved_chunks.

16 Prompt de debogage

Objectif de la phase

Ce prompt doit etre utilise lorsqu'une commande echoue ou lorsqu'une phase ne donne pas le resultat attendu.

```
I got this error in DevPilot AI:

PASTE_FULL_ERROR_HERE

Context:
- Phase: PHASE_NUMBER_AND_NAME
- Command I ran: COMMAND_HERE
- Expected result: EXPECTED_RESULT
- Actual result: ACTUAL_RESULT

Analyze the root cause like a senior backend engineer.

Give me:
1. Exact cause.
2. Why it happened.
3. Files to inspect.
4. Files to modify.
5. Corrected code or exact changes.
6. Commands to rerun.
7. How to verify the fix.

Do not guess.
Do not rewrite unrelated parts.
Keep the fix minimal and clean.
```

17 Prompt de revue d'architecture

Objectif de la phase

Ce prompt est recommande apres chaque groupe de trois ou quatre phases pour verifier la qualite technique du projet.

```
Review the current DevPilot AI codebase as a senior software architect.

Focus on:
1. Clean architecture
2. Security
3. Database design
4. API design
5. Error handling
6. Docker Compose quality
7. Celery reliability
8. Ollama-first configuration
9. Qdrant integration readiness
10. Testability

Give me:
- critical issues
- medium issues
- small improvements
- what to fix now
- what to postpone
- exact refactoring plan
```

Conclusion

Ce document fournit une sequence complete de prompts pour construire DevPilot AI progressivement. Les phases 1 a 8 correspondent a l'infrastructure backend, la securite, les workspaces, l'upload documentaire, le traitement asynchrone, le nettoyage et le chunking. Les phases suivantes ajoutent les embeddings Ollama, Qdrant, la recherche semantique, le chat RAG, l'evaluation et l'agentic workflow.

La meilleure pratique consiste a ne jamais sauter les tests. Chaque phase doit etre validee manuellement avant de passer a la suivante. Cela rend le projet plus fiable, plus facile a expliquer et plus convaincant pour un encadrant ou un jury.